



UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA
Ingeniería en Sistemas de Información

Trabajo de Investigación Grupal - Charla TED **Microservicios de alto tráfico**

Cátedra: Ingeniería y Calidad de Software.

Alumnos:

- Mendez, Guadalupe - 86727
- Montes, Gonzalo - 89603
- Aguirregomezcorra, Martín - 89736
- Grasso, Joaquín - 89952
- Sosa, Ivo - 91078
- Pugliese, Lourdes Belén - 95196
- **Ghibaud, Alan Yair - 97570**
- **Paéz, Miqueas Enry - 98750**
- Derico Farrando, Nazareno - 400744
- Perez, Agustin - 404396
- Bruera, Juan - 431286

Docentes:

- Ing. Covaro Laura.
- Ing. Garnero Constanza.



Indice

Plan de Release.....	2
Charla TED.....	4

Plan de Release

- **Entorno de desarrollo:** Producción en la Nube pública.
 - Esto se debe a que la plataforma posee millones de usuarios concurrentes conectados las 24 horas del día, durante toda la semana, por lo que se requiere de alta disponibilidad y tolerancia a fallos global.
 - También contamos con entornos de: Pre-producción y Testing.
- **Tipo de despliegue:** Despliegue independiente y desacoplado por microservicio.
- **Infraestructura requerida:**
 - **Contenedores de Docker:** Para empaquetar de forma aislada cada microservicio con sus dependencias exactas.
 - **Kubernetes:** Para la administración y escalado automático ante picos de demanda y autorreparación de los contenedores de docker.
 - **API Gateway:** Para gestionar el enrutamiento dinámico del tráfico y la comunicación segura entre microservicios.
- **Artefacto a desplegar:** Imágenes de Docker versionadas unívocamente a través de SCM (Sistema de Gestión de Configuración de Software).
- **Esquema de versionado:** Versionado semántico. Ejemplo: v1.2.1
- **Trazabilidad del SCM:** Cada imagen llevará una etiqueta (tag) inmutable en el registro de contenedores asociada al repositorio. Permitiendo identificar qué código se envía a producción y cuál es la última versión estable identificada en el SCM en caso de realizar un rollback.
- **Despliegue:** El despliegue se ejecuta completamente mediante una **pipeline de CI/CD** (Integración Continua | Despliegue Continuo).
 - **Proceso de despliegue:** Una vez el código pase las pruebas automáticas, la pipeline construye la nueva imagen de Docker, la sube al registro y actualiza los manifiestos de Kubernetes (Archivo YAML) de forma automatizada, eliminando cualquier intervención manual.
- **Dependencias y Prerrequisitos:** Para autorizar que la pipeline avance al entorno productivo, debemos validar los siguientes prerrequisitos en un entorno que no sea de producción (Pre-Producción).
 - **Compatibilidad de contratos (API's):** Verificación de compatibilidad hacia atrás mediante pruebas de contrato de software. Garantizando que ante cualquier modificación en la estructura o el comportamiento de las respuestas de un microservicio no se rompan las dependencias ni el consumo de datos de los servicios preexistentes.
 - **Compatibilidad de la Base de Datos:** Las migraciones de esquemas de datos deben soportar posibles expansiones de datos, modificación de la misma ante re-despliegues y nuevas versiones.
 - **Seguridad y credenciales:** Se deben mantener los tokens y credenciales de producción de forma segura, evitando que no se vean en el código.
- **Horario de Despliegue:** Se realizaría en un horario en donde el tráfico sea el mínimo en base a registros históricos de usuarios. Esto servirá para minimizar tiempos de caída, impactos económicos y usuarios afectados ante fallos por imprevistos de la infraestructura tecnológica.

- **Técnica de despliegue: Canary Deployment**
 - Esto se debe a que nuestro enrutador de tráfico redirige inicialmente un pequeño porcentaje de usuarios hacia la nueva versión del microservicio. Haciendo que el tráfico se incremente de forma escalonada a medida que se validen métricas de estabilidad de la nueva versión del sistema.
- **Monitoreo Activo:** Esto se da al momento del despliegue mediante Canary Deployment.
 - **Tiempo de respuesta (Latencia):** Monitorear que la velocidad de entrega del servicio no se degrade.
 - **Tasa de errores entre API's:** Controlar la presencia de códigos de estados de error que indiquen fallas de comunicación, mediante los registros (logs).
 - **Transacciones de pago exitosas:** Verificar que la pasarela de pagos procese órdenes sin interrupciones.
 - **Calidad de streaming:** Monitorear métricas de bitrate, pérdida de paquetes y almacenamiento en búfer (buffering) reportadas por los clientes de reproducción.
- **Pruebas Post-Despliegue:** Se realizan pruebas automatizadas sobre el tráfico temporal (canario) para confirmar el éxito del release.
 - Algunas de ellas: Flujo de reproducción, Flujo de pagos y Flujo de recomendaciones.
- **Comunicación y validaciones del despliegue**
 - **Antes del despliegue:** Requerimos aprobación obligatoria de la firma digital y aprobación conjunta en la plataforma de gestión del Responsable Técnico y el Product Owner.
 - **Notificaciones:** Aviso automatizado a los canales DevOps, Soporte y Operaciones informando el inicio de la ventana de release.
 - **Después del despliegue:** Notificación de cierre exitoso (o estado de la release) enviada al equipo involucrado junto al reporte de las métricas de monitoreo.
- **Plan de Rollback:** Se dispara en caso de que un despliegue tanto al inicio como al final del mismo utilizando Canary Deployment supere un límite de alerta (Ejemplo: Errores en el despliegue > 1% o caídas en un microservicio), se activa el plan de Rollback, que consta de los siguientes pasos.
 1. **Desvío de Tráfico:** El API Gateway redirige el 100% del tráfico de inmediato a los contenedores que ejecutan la versión anterior **estable**.
 2. **Aislamiento:** Se desvía el tráfico del microservicio fallido a cero, impidiendo que afecte a la infraestructura.
 3. **Preservación del entorno:** Se mantienen los logs y dumps a la base de datos de la versión fallida en un entorno de prueba aislado para que posteriormente sea analizado por el equipo de desarrollo, sin afectar la experiencia del usuario final.

Charla TED

Introducción

[Alan] - Buenas noches, somos estudiantes de la UTN FRC, cursando en la cátedra de ISW y pertenecemos al grupo 5.

Presentación

[Miqueas] - Che es viernes llegan a casa, cansados de recién haber cursado ISW, se preparan algo de comer y ponen su serie favorita, ya entraron en el Clímax de la película y justo en la mejor parte ¡PUM! se corta la transmisión. Porque al equipo de desarrollo se le ocurrió darle el botón de deploy un VIERNES, y ni siquiera era un gran cambio, era una funcionalidad que supuestamente optimizaba un flujo de la transmisión.

Causando congelación en las pantallas de los usuarios,

generando un cambio en la estructura de datos haciendo que los microservicios no se puedan comunicar debido a problemas de formato, haciendo que colapse el sistema debido a múltiples peticiones. También falló la pasarela de pago, causando grandes pérdidas, lo que termina dando como resultado miles de usuarios frustrados quejándose por el mal servicio.los

[Miqueas] - Ahora, bien ¿Cómo nos evitamos este lío? además de no hacer deploy un viernes...

[Alan] - Para responder a esta pregunta necesitamos un Plan de Release. No tenemos que ser el Messi de los Deploy, pero hay conceptos básicos de los cuales debemos tener noción.

Para empezar, tenemos que entender dónde vamos a desplegar nuestro sistema.

Ya que estamos ante una plataforma de streaming con millones de usuarios conectados las 24 horas, nuestro entorno de producción debería ser en la nube.

Y al trabajar con microservicios, los son los que desplegaríamos, no tenemos una única aplicación monolítica, sino decenas de servicios independientes funcionando al mismo tiempo: catálogo, recomendaciones, pagos, historial de usuarios y muchos más.

Para lo que requerimos de infraestructura que nos permita administrar todos esos servicios de manera eficiente. Una alternativa muy utilizada es ejecutar los microservicios en contenedores Docker que tienen lo necesario para que los microservicios funcionen y administrados por Kubernetes se encargaría de distribuirlas en servidores, reiniciarlas ante fallos y escalarlas ante aumento de usuarios.

[Miqueas] - O sea que tenemos un montón de servicios corriendo al mismo tiempo...

¿Y cómo sabemos exactamente qué estamos desplegando para no mandarnos ninguna durante el desarrollo?

[Alan] - Temita de parcial 1 eh, vamos a usar SCM.

Esta técnica nos va a permitir identificar exactamente qué versión de cada microservicio estamos desplegando mediante etiquetas y versiones únicas.

De esta manera sabemos qué imagen de Docker versionada llevamos a producción y, más importante todavía, cuál es la última versión estable por si necesitamos volver atrás.

Porque créanme... cuando todo sale mal, lo que más queremos saber es que paso. [i'm fine]

[Miqueas] - Está perfecto para no romper nada durante el desarrollo.

¿Pero qué pasa cuando llega el momento de desplegar?

[Alan] - Antes de desplegar tenemos varios prerequisites que validar.

Tenemos que asegurarnos de que las interfaces y versiones de las APIs mantengan compatibilidad entre servicios, que las dependencias entre microservicios funcionen correctamente y que las credenciales necesarias estén configuradas de forma segura.

[Miqueas] - Che alan yo no tengo muchas ganas de laburar, y para evitar errores de capa 8 y no mandar moco, usemos una Pipeline CI/CD (Integración Continua y Despliegue continuo), que automatiza validaciones, pruebas y controles antes de permitir que una nueva versión llegue a producción.

[Alan] - Todo esto se da por DevOps.

Ya que desplegar no es tarea de una sola persona. Desarrollo, Operaciones y Calidad trabajan juntos para que cada cambio llegue a producción de manera segura.

La Pipeline CI/CD que nombramos es justamente una de las prácticas que nos permite automatizar ese proceso y reducir riesgos.

[Miqueas] - O sea que no depende de que alguien se acuerde de hacer las cosas bien un viernes a la noche.

[Alan] - Exactamente. La idea es que el proceso sea confiable aunque nosotros no lo seamos.

[Miqueas] - Bueno, ya tenemos los servicios identificados, la infraestructura lista y una Pipeline que controla todo.

Pero seguimos teniendo un problema.

Tenemos millones de usuarios mirando series y películas.

¿Cómo hacemos para desplegar sin que se caiga la plataforma?

[Alan] - Lo ideal sería usar una estrategia de despliegue gradual para reducir riesgos y que nos permita validar que todo funciona correctamente.

Para eso vamos a usar Canary Deployment.

En lugar de reemplazar inmediatamente la versión actual, liberamos la nueva versión solamente para un pequeño porcentaje de usuarios, porque si algo explota, explota para muy pocos.

[Miqueas] - Y mientras esos usuarios utilizan la nueva versión, verificamos que los microservicios sigan comunicándose correctamente, que los pagos funcionen y que la calidad del streaming se mantenga.

Si todo sale bien, aumentamos gradualmente el tráfico de usuarios hacia la nueva versión hasta llegar al 100%.

[Alan] - A su vez, tenemos en cuenta que no vamos a desplegar un viernes a la noche, vamos a hacerlo cuando menor tráfico de usuarios tengamos, minimizando el impacto en caso de que haya algún inconveniente.

[Miqueas] - Durante el despliegue tenemos que monitorear métricas para prevenir una posible caída de producción, estas son:

- Tiempo de respuesta.
- Transacciones de pago exitosas.
- Errores entre API's.
- Calidad del streaming.

[Alan] - Para comprobar que desplegamos todo bien, vamos a realizar pruebas post-despliegue como:

- Reproducir contenido.
- Procesar pagos.
- Obtener recomendaciones.

Y muchas otras más, con las que podamos considerar exitoso nuestro release.

[Miqueas] - Pero antes de decidir cuando va a salir todo esto, debemos comunicar a todo el equipo involucrado para aprobar la release, tanto del lado del responsable técnico como el product owner.

Así todos saben los cambios realizados y pueden actuar ante cualquier problema.

Pero ¿que pasa si sale todo mal?

[Alan] - Acá entra el Rollback.

Al implementar SCM sabemos cuál fue la última versión estable, por lo que ante problemas, detenemos el tráfico hacia la nueva versión y volvemos a la versión anterior.

[Miqueas] - Lo importante de todo esto es que el usuario no se acuerda de la página cuando anda, se acuerda cuando algo falla o tiene un error.



[Alan] - Por lo que desplegar software no es solamente subir código, tenemos que gestionar riesgos, coordinar equipos y proteger la experiencia de millones de usuarios.

[Miqueas] - Porque en una plataforma de streaming, nadie se da cuenta de un buen plan de release.

[Alan] - Todos se dan cuenta, ante un mal plan de release.

Muchas gracias por su atención !



Presentación

https://docs.google.com/presentation/d/1YfxRnqPDw9hxWbb9CZ7OSjeO7FHH8oQVz3cLHDskfus/edit?slide=id.g3ed48f773ff_0_0#slide=id.g3ed48f773ff_0_0